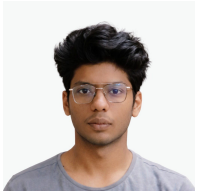
PROJECT AND TEAM INFORMATION

Project Title

CarInfo – Smart Car Infotainment Resource Manager

Student / Team Information

Team Name: *Team #*	*NexGen*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Bisht,Karthik –24011265* *24011265@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Arif,Mohd – 240111049* *240111049@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Shiwani – 240221796* *240221796@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Shanya – 240221725* *240221725@geu.ac.in* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Modern car infotainment systems really need to multitask - they're meant to play music, navigate, recognize our voices, communicate via Bluetooth, make phone calls, and run various background services all at once. When lots of tasks compete for the CPU and memory resources at the same time, the entire system may experience extremely high resource utilization, delayed responses, or even screen lag.
The main aim of CarInfo - Smart Car Infotainment Resource Manager is creating a resource management system that monitors the infotainment processes themselves, identifies which tasks are really consuming resources, and gives them an appropriate process priority based on their importance. Instead of just telling us that the infotainment system is slow, CarInfo will identify the specific processes responsible for using up resources like CPU time and RAM usage - and then decide which one should have a higher priority itself!
The project also aims to demonstrate just how some operating system concepts - such as process management, process priority, CPU scheduling, memory management, multithreading, and resource allocation - can actually be put into practice within a real-world infotainment workload. All the system data collected will be stored in a PostgreSQL database - so we'll be able to see resource use and performance trends over time itself.

State of the Art / Current solution (1 pt)

Current research and automotive systems are focused on managing resources and creating schedules for vehicle computing environments. A few techniques concentrate on sharing and controlling infotainment resources themselves - whilst others focus on resource planning, automotive operating systems or task scheduling for automotive processors.
However, these approaches primarily focus on only a part of automotive resource management itself. A resource control system might focus on sharing entertainment resources - while automotive scheduling research is more concerned with task latency, multicore scheduling, or even low-level resource allocation itself. These systems rarely integrate operating system-level process and resource monitoring into a database-management-system-based (DBMS) history and performance-analysis dashboard for our everyday entertainment tasks.
CarInfo specifically targets infotainment activities such as navigation, voice commands, music, Bluetooth, phone calls and background services themselves. It combines actual OS-level monitoring and resource management itself with PostgreSQL-based storage and analysis. This lets the system really observe CPU usage, memory usage, process status, execution/response time, scheduling order and performance history all within a single system itself.
Therefore the proposed system provides a student-level working implementation demonstrating how OS resource management itself and DBMS-based performance analysis will work together in order to improve infotainment responsiveness itself.

Project Goals and Milestones (2 pts)

PROJECT GOALS

Process management: monitor your car's infotainment processes - such as music, navigation, phone calls, Bluetooth, speech recognition, and background applications.
Process monitoring: track each process's ID number, CPU usage, memory consumption, status, and response time itself.
Process priority: establish priorities based on importance levels - emergency/voice controls holding the top spot, navigation/phone calls ranked second, music/Bluetooth set at medium level, while background functions receive the lowest priority setting itself.
CPU Scheduling: design and then test both the priority-based and round-robin scheduling algorithms individually yourselves.
Scheduling analysis: calculate the waiting time, turnaround time, response time, CPU utilization, and even the order in which tasks are executed itself.
Memory management: monitor your vehicle's RAM usage closely - identifying any process that consumes excessively large amounts of memory itself all the time.
Multithreading: make use of separate C++ threads for the CPU, RAM, process monitoring, and also database/server communications itself directly every time.
Resource allocation: dynamically allocate both the CPU and RAM resources based on the process priority, requirements, as well as your present workload itself all the time too.
Load management: automatically recognize when your vehicle's system is overwhelmed with a heavy load - then prioritize those extremely urgent infotainment tasks itself instantly all the time.
DBMS analysis: save relevant details regarding each process, its resource usage, allocations made, key events, and even performance metrics into a PostgreSQL database for future historical analysis purposes itself every now and then.
Dashboard display: show you the real-time usage figures of the CPU and RAM, the list of currently running processes, their respective priorities, as well as any important system alerts or warnings in real-time itself continuously always.

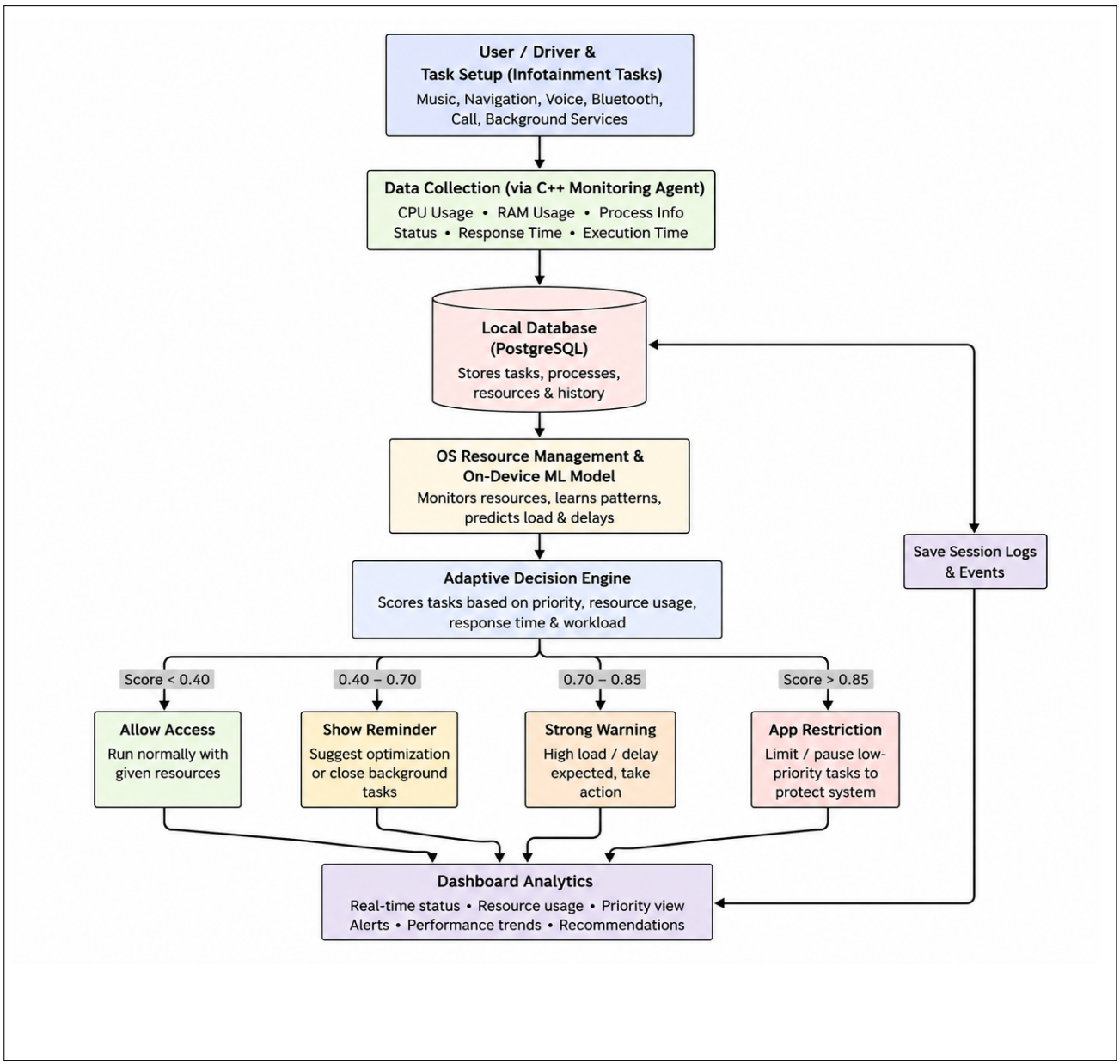
MILESTONES

System design and setup - determine the overall framework and develop the entertainment functions, OS resources, database tables, and communication paths.
OS monitoring module - create the C++ resource monitoring component to collect data on CPU, RAM, processes, and execution-related info directly from the operating system itself.
Process priority and scheduling module - establish process-priority management and compare priority scheduling with round-robin scheduling using different performance metrics for comparison purposes.
Memory and resource management - monitor memory usage and create resource allocation and load management logic depending on system conditions and priorities of tasks themselves.
Multithreading module - develop separate monitoring and communication threads for CPU, RAM, processes, and also server/database communication itself.
PostgreSQL integration - store all collected resource usage, process details, task history, allocation information, system events, and performance events right in the database itself.
Dashboard development - develop a dashboard displaying real-time system status and historical performance information presented in a very clear way.
Testing and performance analysis - test out many different entertainment loads and assess the performance of our scheduling algorithm.

Project Approach (3 pts)

The system has a very modular structure that combines concepts from operating systems, C++, PostgreSQL - and even a monitoring dashboard.
Infotainment tasks include managing procedures such as music, navigation, speech recognition, Bluetooth, phone calls and background applications.
Process monitoring is achieved by means of a C++ agent that checks on the process ID, CPU usage, memory usage, process status and how long it takes to respond.
Resource management involves determining priorities for each process according to its level of importance - emergencies and voice commands get top priority.
CPU scheduling will actually implement and compare both priority scheduling and round-robin algorithms using metrics such as waiting time, turnaround time, response time, CPU utilization and the order in which jobs are scheduled.
Memory management involves tracking RAM usage at all times and identifying those processes using too much memory itself. We employ multithreading extensively here - separate C++ threads monitor the CPU and RAM, check other processes and handle our database communications too.
Load management identifies cases when the CPU or RAM are under heavy load and then adjusts the priority levels of our most critical infotainment functions accordingly - so they get the necessary resources.
Our PostgreSQL database keeps a record of all relevant process, resource, task, allocation, event and performance data - for later analysis purposes no less.
Finally, our dashboard gives you live readings of current CPU and RAM usage, lists out the active processes along with their respective priorities, offers a bit more performance info - and also warns you about any potential issues within the system itself in real-time!

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

Expected results from CarInfo include an operational intelligent infotainment resource management system demonstrating how Operating System and DBMS concepts are effectively combined. This project is geared towards creating an Agent that monitors both infotainment as well as system process along with their resource consumptions. In addition to this, there is also the management of processes at the Operating System level whereby it is possible to control the priority given to a process and apply or study various techniques of scheduling like Priority Scheduling or Round Robin.
It determines key metrics including waiting time, turnaround time, response time, CPU utilization, and scheduling order. The monitoring system extends to keeping record of CPU and memory consumption while enhancing inter-process communication with the server and database. There is also a resource allocation module which assigns resources depending on the process requirements, priority and system load. Processes, tasks, resource allocation, events and performance history related information is stored in PostgreSQL.
In the end, a user interface shows data such as CPU usage, RAM usage, active tasks, task priorities, system alerts, and performance details through which we can draw conclusions like which processes require most resources, how does the workload vary with time, etc.? and what impact different scheduling algorithms have on system performance.

Assumptions

The infotainment system will feature numerous processes, all operating simultaneously.
Each process will have extremely well-defined CPU, memory, and priority specifications predefined for it.
Critical functions such as emergency alerts and voice commands demand significantly higher priority levels be assigned to them.
CPU scheduling is currently under evaluation using both Priority Scheduling and Round Robin methodologies.
The system itself possesses more than sufficient CPU and RAM resources for regular daily usage.
PostgreSQL will be utilized for storing process and performance-related information data.
System monitoring data is retrieved at regular intervals directly from the underlying operating system itself.
Your dashboard will display the most up-to-date available system and process details that we've managed to gather thus far. This project is focusing its efforts mainly on simulating and monitoring - rather than directly controlling any life-critical vehicle systems themselves.

References

- *IoT-Based Resource Control for In-Vehicle Infotainment Services: Design and Experimentation, Sensors, 2019. Available at: MDPI – Sensors*
- *AUTOSAR Operating System – Automotive OS Standard, AUTOSAR. Available at: AUTOSAR Operating System Specification*
- *RunSoC: Scheduling and Allocation of Automotive Software Tasks to SoC Partitions, Journal of Systems and Software, 2026. DOI:10.1016/j.jss.2026.112830.*
- *AA Torque – Performance Monitor for Android Auto, GitHub project. Available at: AA Torque GitHub Repository*
- *PostgreSQL Documentation, PostgreSQL Global Development Group. Available at: PostgreSQL Documentation*